Estimating a probability from data: three coin-flip scenarios

The lecture opens with a thought experiment about how observations should update our belief in an unknown probability $p = P(\text{heads})$.

Scenario 1 — two flips, two heads. Under a perfectly fair coin this happens with probability $(1/2)^2 = 1/4$: completely unremarkable. The data carry essentially no information about the coin, so the next flip remains fifty-fifty.

Scenario 2 — one hundred flips, all heads. Here intuition flips. A fair coin producing heads 100 times in a row has probability $(1/2)^{100}$, which is vanishingly small; a far simpler explanation is that the coin itself is not fair — perhaps it has two heads. Crucially, the data have now changed our belief about *the coin*, not merely about the next flip.

Scenario 3 — one hundred flips, fifty-two heads and forty-eight tails. Given only this data, the natural estimate of the probability of heads is the observed fraction,

$$\hat{p} = \frac{52}{100} = 0.52.$$

But — and this is the crucial point — our *confidence* in that estimate should be low. A near-even split is exactly the kind of result randomness produces all the time; the data are entirely consistent with a fair coin. So we hold an estimate we do not much trust. The lesson: an estimate and justified confidence in that estimate are two separate things.

What determines confidence: sample size and variance

Why were we suspicious after a hundred heads yet tentative about 0.52? Confidence in an estimate depends on two properties of the data:

1. **Sample size** — one hundred observations versus two. The more flips we have seen, the more we are willing to believe what they say.
2. **Variance of the sample** — all heads versus a mixed outcome. When every observation is identical, the data are telling us something dramatic; when observations are spread out and mixed, each one is far less informative.

The operating rule: **as the variance grows, we need larger samples to achieve the same degree of confidence.** Noisy data demand more of it before we believe our conclusions. This framing sets up the central problem of the lecture — inferring an unknown probability (such as a coin's fairness) from random outcomes — and motivates building trustworthy simulation machinery to attack it.

Roulette: a system whose probabilities we know exactly

The next example is roulette, and its value lies in a special property: **there is no need to simulate it**, because the answers can be computed exactly on paper. The mechanism is fully specified — a croupier spins the wheel in one direction and a ball in the opposite direction around a tilted circular track; the ball loses momentum, passes through deflectors, and settles into one of the colored, numbered pockets, and winning bets are paid. Because the wheel has a known, fixed layout of pockets, the probability of any outcome is just a counting exercise over those pockets.

The betting structure inherits this exactness. Players choose among bets on a single number, groupings of numbers, red or black, odd or even, or high or low. Wikipedia distinguishes **inside bets** (an exact number, or a small group of adjacent numbers on the layout) from **outside bets** (larger groups defined by properties such as color or parity), and notes that the payout odds for each type of bet are based on its probability. Historically, the casino's edge came from reserved bank slots — the zero and double zero, described in an 1801 French novel as the slots "whence it derives its sole mathematical advantage" — and wheel variants differ precisely in these pockets: the Blanc brothers introduced the single-zero wheel at Bad Homburg in 1843, while the double-zero wheel remained dominant in America.

Why simulate anyway: Monte Carlo as validated machinery

If roulette's probabilities are computable by hand, why write a simulation at all? Because roulette serves as a **test case**: if simulation code reproduces the known exact probabilities, that gives us reason to trust the code — and once the machinery is trusted, we can aim it at problems where the answers are *not* obvious, like inferring the fairness of the coin above.

This is exactly the logic of the **Monte Carlo method**: a broad class of computational algorithms based on repeated random sampling to obtain numerical results, conceptualized by Stanisław Ulam (the name comes from the Monte Carlo Casino, inspired by his uncle's gambling habits). The underlying idea is to *use randomness to solve deterministic problems.* Monte Carlo methods are mainly applied to three problem classes — optimization, numerical integration, and generating draws from probability distributions — and to modeling phenomena with significant input uncertainty, such as risk assessments for nuclear power plants. They provide approximate solutions precisely where problems are too complex for mathematical analysis.

Two pieces of the Wikipedia account explain why the approach works and what it demands:

- **Why it converges:** by the law of large numbers, quantities described by the expected value of a random variable can be approximated by the empirical mean of independent samples. Repeating a simulated experiment many times and averaging therefore approaches the true probability — the same principle that makes the classic demonstration work, where grains scattered randomly over a unit square containing a quadrant (area ratio $\pi/4$) yield an estimate of π from the fraction landing inside the quadrant.
- **What it requires:** large amounts of random numbers (which is why pseudorandom number generators, far quicker than old random-number tables, were essential), and care about the trade-off between accuracy and computational cost, the reliability of the random number generators themselves, and — critically — the **verification and validation** of results. That last item is precisely the role the roulette test case plays: validating simulation output against ground truth before trusting it where no ground truth exists.

Monte Carlo Methods: Using Randomness to Solve Deterministic Problems

Monte Carlo methods are a broad class of computational algorithms based on repeated random sampling to obtain numerical results. They were conceptualized by Polish mathematician Stanisław Ulam, who was inspired by his uncle's gambling habits at the Monte Carlo Casino in Monaco — hence the name. The underlying concept is to use randomness to solve deterministic problems.

These methods are mainly used in three distinct problem classes: - **Optimization** - **Numerical integration** - **Non-uniform random variate generation**

They are particularly valuable for modeling phenomena with significant input uncertainties, such as risk assessments for nuclear power plants. Monte Carlo methods can provide approximate solutions to problems too complex for mathematical analysis, which is precisely why we turn to them here rather than grinding through probability calculus.

A classic illustration is estimating π: consider a quadrant (circular sector) inscribed in a unit square. The ratio of their areas is $\pi/4$. One scatters random points over the square, tests whether each falls within the quadrant, and aggregates the results to approximate π. This exemplifies the general pattern: define a domain, generate random inputs, perform a computation on each input, and aggregate.

Two important considerations apply: Monte Carlo methods require large amounts of random numbers (benefiting greatly from pseudorandom number generators), and they are most useful when it is difficult or impossible to use other approaches. By the **law of large numbers**, integrals described by the expected value of some random variable can be approximated by taking the empirical mean (sample mean) of independent samples. When the distribution is parameterized, mathematicians often use Markov chain Monte Carlo (MCMC) samplers, designing a judicious Markov chain whose stationary distribution matches the target distribution; by the ergodic theorem, the empirical measures of the sampler's states approximate that stationary distribution.

Limitations include the trade-off between accuracy and computational cost, the curse of dimensionality, reliability of random number generators, and verification/validation challenges. Despite these, Monte Carlo methods are recognized among the most influential ideas of the 20th century, enabling breakthroughs across physics, chemistry, biology, statistics, AI, finance, cryptography, and even social sciences.

Roulette: History and Mechanics

Roulette (French for "little wheel") is a casino game likely developed from the Italian game Biribi. A player may bet on a single number, groupings of numbers, red or black color, odd/even, or high/low. To determine the winning number, a croupier spins the wheel in one direction and spins a ball in the opposite direction around a tilted circular track on the outer edge. The ball loses momentum, passes through deflectors, and falls into one of the colored, numbered pockets.

Historically, Blaise Pascal may have introduced a primitive form in his 17th-century search for a perpetual motion machine. The mechanism combines a gaming wheel invented in 1720 with Biribi. The game has been played in its present form since at least 1796 in Paris, as described in Jaques Lablee's novel *La Roulette, ou le Jour*, which noted "exactly two slots reserved for the bank, whence it derives its sole mathematical advantage" — the zero and double zero. In 1843, François and Louis Blanc introduced the single-zero wheel in Bad Homburg to compete against double-zero casinos. When Germany abolished gambling in the 1860s, the Blanc family moved to Monte Carlo, establishing the single-zero wheel as the premier game there, while the American double-zero wheel remained dominant in the US.

Betting options divide into: - **Inside bets**: selecting an exact number or a small adjacent group - **Outside bets**: larger groups based on properties like color or parity

Payout odds for each type are based on probability. Table minimums and maximums usually apply separately to inside and outside bets per spin.

Simulating a Fair Roulette Wheel

We build a `FairRoulette` class to model the game computationally. The `__init__` method constructs the wheel:

- Creates an empty list called `pockets`, then appends integers 1 through 36 (`for i in range(1, 37)`), giving 36 pockets.
- Sets `self.ball = None` since the ball hasn't landed yet.
- Sets `self.pocketOdds = len(self.pockets) - 1`, i.e., $36 - 1 = 35$. This is the payout multiplier: winning a pocket bet pays 35 times your stake, matching real roulette's 35-to-1 odds.

The `spin` method uses `random.choice` to select one pocket uniformly at random — that's where the ball lands. The `betPocket` method takes a target pocket and an amount, compares `str(pocket)` to `str(self.ball)` (converting both to strings for comparison): if they match, you win `amt * pocketOdds`; otherwise you lose and the method returns `-amt`. Finally, `__str__` returns `'Fair Roulette'`.

Why call it "fair"? Because every pocket is equally likely — there is no house edge built in, unlike a real casino wheel which includes extra green zero pockets that give the bank its mathematical advantage.

Running the Simulation: Convergence to Expected Value

To find out what happens when we actually play, we simulate rather than derive analytically. The function `playRoulette(game, numSpins, pocket, bet, toPrint)` initializes total winnings `totPocket = 0`, then for each of `numSpins` iterations calls `game.spin()` and adds `game.betPocket(pocket, bet)` to the total. If printing is enabled, it invokes `__str__` (printing "Fair Roulette") and reports the expected return as a percentage:

$$\text{expected return} = \frac{100 \times \text{totPocket}}{\text{numSpins}}$$

The function returns `totPocket / numSpins`, the return per spin.

We instantiate the game and run it for two sample sizes — 100 spins (small) and 1,000,000 spins (huge) — three times each, betting on pocket 2 with one unit per spin. Why three repetitions? Because with few spins, the answer jumps around dramatically.

Results with 100 spins: - Run 1: -100.0% - Run 2: $+44.0\%$ - Run 3: -28.0%

The spread runs from losing everything to gaining 44 percent — wildly inconsistent.

Results with 1,000,000 spins: - Run 1: -0.046% - Run 2: $+0.602\%$ - Run 3: $+0.7964\%$

All three cluster tightly around zero. And zero is exactly what theory predicts for a fair wheel: with 36 equally likely pockets and 35-to-1 payout, the expected value per spin is

$$E = \frac{1}{36}(35) + \frac{35}{36}(-1) = \frac{35 - 35}{36} = 0$$

This demonstrates the law of large numbers empirically: as the number of independent trials grows, the empirical mean converges toward the theoretical expected value. Small samples exhibit high variance; large samples stabilize around the true mean.

Regression Toward the Mean

Regression toward the mean (also called *reversion to the mean* or, colorfully, *reversion to mediocrity*) is the phenomenon where, if one sample of a random variable is extreme, the next sampling of the same random variable is likely to be closer to its mean. The professor's roulette example makes the mechanics concrete:

- Spin a fair wheel 10 times and get 100% red. Each spin is effectively a coin flip, so the probability of ten straight reds is
$$\left(\tfrac{1}{2}\right)^{10} = \frac{1}{1024},$$
a genuinely extreme event.
- The key point: it is *likely* that the next 10 spins contain fewer than 10 reds — **not** because the wheel "owes" you black (the wheel has no memory), but simply because the expected number of reds in 10 spins is only 5. Your streak was an outlier; the next batch will probably be ordinary.
- Consequently, the average over all 20 spins lands closer to the expected 50% red than to the freakish 100% of the first batch. There is no mysterious balancing force in the universe — regression is just what happens arithmetically when you average an extreme outcome together with typical ones.

This is worth emphasizing because the intuition goes wrong so easily: as the law-of-large-numbers literature stresses, there is **no principle** that a small number of observations will coincide with the expected value, nor that a streak of one outcome will be immediately "balanced" by the others (this mistaken belief is the *gambler's fallacy*). Even in a long sequence of fair coin flips where the *proportion* of heads converges to $\frac{1}{2}$, the absolute difference between the number of heads and tails typically *grows* — it is the ratio of that difference to the number of flips that approaches zero. Convergence is relative, not a cosmic correction.

How strong the effect is depends on the underlying distributions. When successive samples come from the same distribution, regression is statistically likely to occur; when there are genuine differences in the underlying distributions, it may occur weakly or not at all. The students-taking-a-test example splits the cases cleanly:

- *Pure luck:* everyone guesses randomly on a 100-item true/false test, so scores are i.i.d. with mean 50. Selecting the top 10% and retesting, their mean returns to ~50 — regression all the way back. No matter what a student scored initially, the best prediction of their second score is 50.
- *Pure skill:* if answers involved no luck at all, students would score identically on both tests, and there would be no regression.

- *Realistic mix (skill + luck):* the above-average group contains skilled students who didn't have bad luck *and* unskilled students who were extremely lucky. On a retest, the unskilled are unlikely to repeat their lucky break, while the skilled now have a chance at bad luck — so high scorers tend to slip somewhat. Symmetrically, unlucky low scorers tend to improve. The larger the role of luck in producing an extreme result, the less likely that luck repeats.

This last case explains familiar patterns: a championship team regresses next season to the extent its title reflected luck (favorable draws, rivals' scandals) rather than skill; a company's unusually profitable quarter is likely followed by a weaker one even with nothing changed; rookie baseball stars hit the "sophomore slump"; and the "Sports Illustrated cover jinx" is regression misread as causation — exceptional performance earns the cover, and exceptional performance is naturally followed by more mediocre performance.

A practical lesson for experimental design: whenever you deliberately select the most extreme cases, follow-up checks are essential, because those extremes may be genuine, pure statistical noise, or a mixture — and jumping to conclusions about them invites false findings.

The very name of the technique comes from Francis Galton's studies of heredity: he observed that adult children's heights deviate *less* from the population mean than their parents' heights do, and this "regression" of extremes toward the middle gave regression analysis its name.

The House Edge: Fair, European, and American Wheels

Everything above assumed a *fair* wheel — but casinos are not in the business of fairness. The edge lives in the **green pockets**: when the ball lands on green, every red/black bet loses. The American wheel carries two green pockets (0 and 00); the European wheel carries one (0). That little bit of green is how the casino pays for the building — historically, an 18th-century description of the Paris game already noted the bank's reserved slots as the source of its "sole mathematical advantage." The single-zero wheel was introduced by the Blanc brothers in 1843 (at Bad Homburg) as a competitive draw, became the signature game of Monte Carlo, and remains standard worldwide except in the Americas, where the double-zero wheel stayed dominant.

These three games map beautifully onto **inheritance**, with each subclass adding exactly one feature to the model:

```
class EuRoulette(FairRoulette):
    def __init__(self):
        FairRoulette.__init__(self)       # build the fair wheel...
        self.pockets.append('0')          # ...then add the green zero
    def __str__(self):
        return 'European Roulette'

class AmRoulette(EuRoulette):
    def __init__(self):
        EuRoulette.__init__(self)         # fair wheel + '0' already done
        self.pockets.append('00')         # ...then add the second green pocket
    def __str__(self):
        return 'American Roulette'
```

Note the chain: `AmRoulette` inherits from `EuRoulette`, which inherits from `FairRoulette`. Calling `EuRoulette.__init__` already builds the fair wheel *and* appends `'0'`, so constructing an American wheel is literally "the European wheel plus one more green pocket." Each level of the hierarchy contributes exactly one thing — a tidy encoding of how the real games differ.

Watching Convergence Happen: Simulation Results

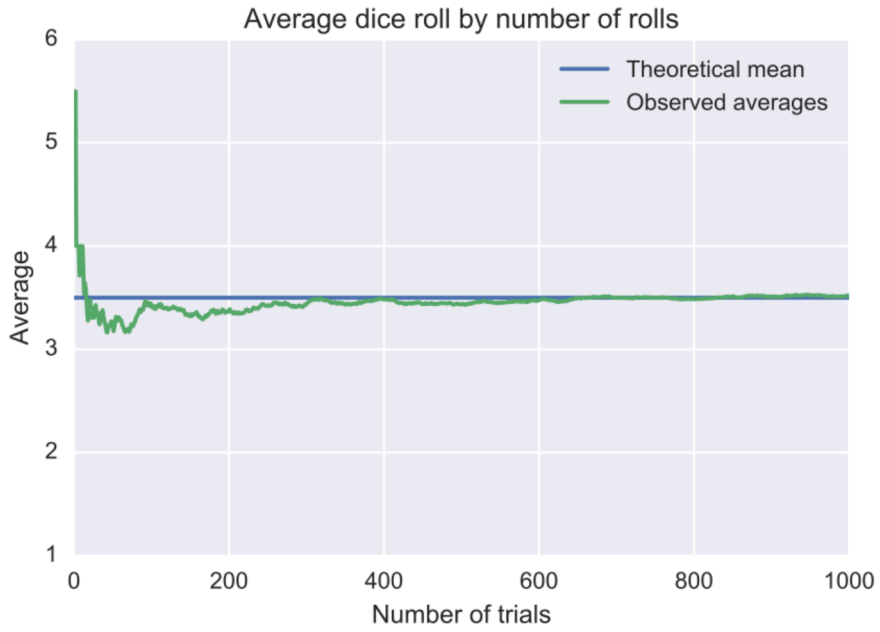
Running 20 trials of n spins each and computing the expected return shows the law of large numbers doing its work:

Spins per trial	Fair	European	American
1,000	+6.56%	−2.26%	−8.92%
10,000	−1.234%	−4.168%	−5.752%
100,000	+0.8144%	−2.6506%	−5.113%
1,000,000	−0.0723%	−2.7329%	−5.212%

Read the table as a story:

- **Small samples lie.** At 1,000 spins the *fair* wheel shows +6.56% — a positive return for a game whose true expected return is zero. With so few observations we are entirely at the mercy of luck; there is no principle forcing a small sample to match the expected value.
- **Convergence is real but gradual.** By 10,000 spins the fair wheel has gone negative, yet at 100,000 it briefly reads +0.81% again before settling at −0.07% by a million spins. The estimates stabilize only as the sample grows.
- **The house edges emerge.** The European wheel settles near −2.7% and the American near −5.2% — roughly *double*, thanks to that one extra green pocket. As the Wikipedia account puts it, a casino may lose money on any single spin, but over a large number of spins its earnings tend toward a predictable percentage, and any player's winning streak is eventually overcome by the parameters of the game itself.

Formally, the **law of large numbers** states that for independent, identically distributed samples, the sample mean converges to the true mean (if it exists): the average of many rolls of a fair die approaches $\frac{1+2+3+4+5+6}{6} = 3.5$, and the empirical success rate of repeated Bernoulli trials converges to the theoretical probability. The same principle underlies Monte Carlo methods, which obtain numerical results purely by repeated random sampling — often the only feasible approach — with accuracy improving as repetitions increase. Historically, Cardano asserted the idea without proof; Jacob Bernoulli spent over twenty years proving the binary case, publishing it as his "golden theorem" in *Ars Conjectandi* (1713); Poisson christened it "la loi des grands nombres" in 1837.



Source: Wikipedia, article "Law of large numbers".

How Many Samples? Accuracy, Confidence, and Variability

The simulation raises the question head-on: **when sampling a space of possible outcomes, it is never possible to *guarantee* perfect accuracy.** An estimate can certainly land exactly on the right answer — the fair wheel did show +0.81% once — but that precision cannot be guaranteed in advance. So the operative question becomes: *how many samples do we need before we can have justified confidence in our answer?*

The answer depends on the **variability of the underlying distribution**:

- If outcomes are tightly clustered around the mean, a modest sample already tells you a great deal.
- If outcomes are wildly spread out, far more samples are needed to reach the same level of confidence.

Two cautions from the theory bound what sampling can ever achieve:

1. **Heavy tails break convergence entirely.** For distributions like the Cauchy (which has no expected value) or Pareto with shape parameter $\alpha < 1$ (infinite expectation), the average of n samples does *not* converge as n grows — for the Cauchy case, the average of n draws has the same distribution as a single draw. The law of large numbers requires a well-defined mean to converge to.
2. **Sampling cannot cure bias.** If the trials embed a selection bias — common in human economic behavior — increasing the number of trials leaves the bias intact. More data reproduces the bias more faithfully; it does not remove it.

Within those limits, though, the law guarantees stable long-term results, and quantifying exactly how sample size trades off against variability to produce justified confidence is the program for the rest of the course.

Reading the Simulation Results: When Does the Truth Emerge?

The payoff of all the machinery built so far is a set of concrete numbers. In each experiment the bet is on a single pocket, run as **20 independent trials**, with the reported expected return given as an estimate plus-or-minus its 95% confidence interval:

Spins per trial	Fair roulette	European roulette	American roulette
1,000	$+3.68\% \pm 27.189\%$	$-5.5\% \pm 35.042\%$	$-4.24\% \pm 26.494\%$
100,000	$+0.125\% \pm 3.999\%$	$-3.313\% \pm 3.515\%$	$-5.594\% \pm 4.287\%$
1,000,000	$+0.012\% \pm 0.846\%$	$-2.679\% \pm 0.948\%$	$-5.176\% \pm 1.214\%$

At 1,000 spins the confidence intervals are **enormous** — the fair-wheel interval alone spans roughly -23% to $+31\%$. All three intervals overlap heavily, so with only 1,000 spins we **cannot distinguish the three games from one another**: the noise completely swamps the signal.

By 1,000,000 spins the picture inverts. The intervals are tight enough to **separate the games cleanly**: the fair game is clearly centered near zero, the European game is clearly losing about 2.7%, and the American game is clearly worse, around -5%. With enough samples, the **law of large numbers** wins and the intervals tighten enough to reveal the truth.

Notice the arithmetic hidden in the table: multiplying the number of spins by 100 (from 1,000 to 100,000) shrank the interval widths by roughly a factor of 10 ($27.19\% \to 3.999\%$). This is not an accident. Averaging n independent observations divides the variance of the estimate by n, so the typical error — and hence the interval width — shrinks only like $\sqrt{n}$:

$$\text{width} \propto \frac{1}{\sqrt{n}}$$

This is why precision is **expensive**: going from 100,000 to 1,000,000 spins ($10\times$ more work) bought only a modest improvement, and cutting the width in half would require $100\times$ more samples.

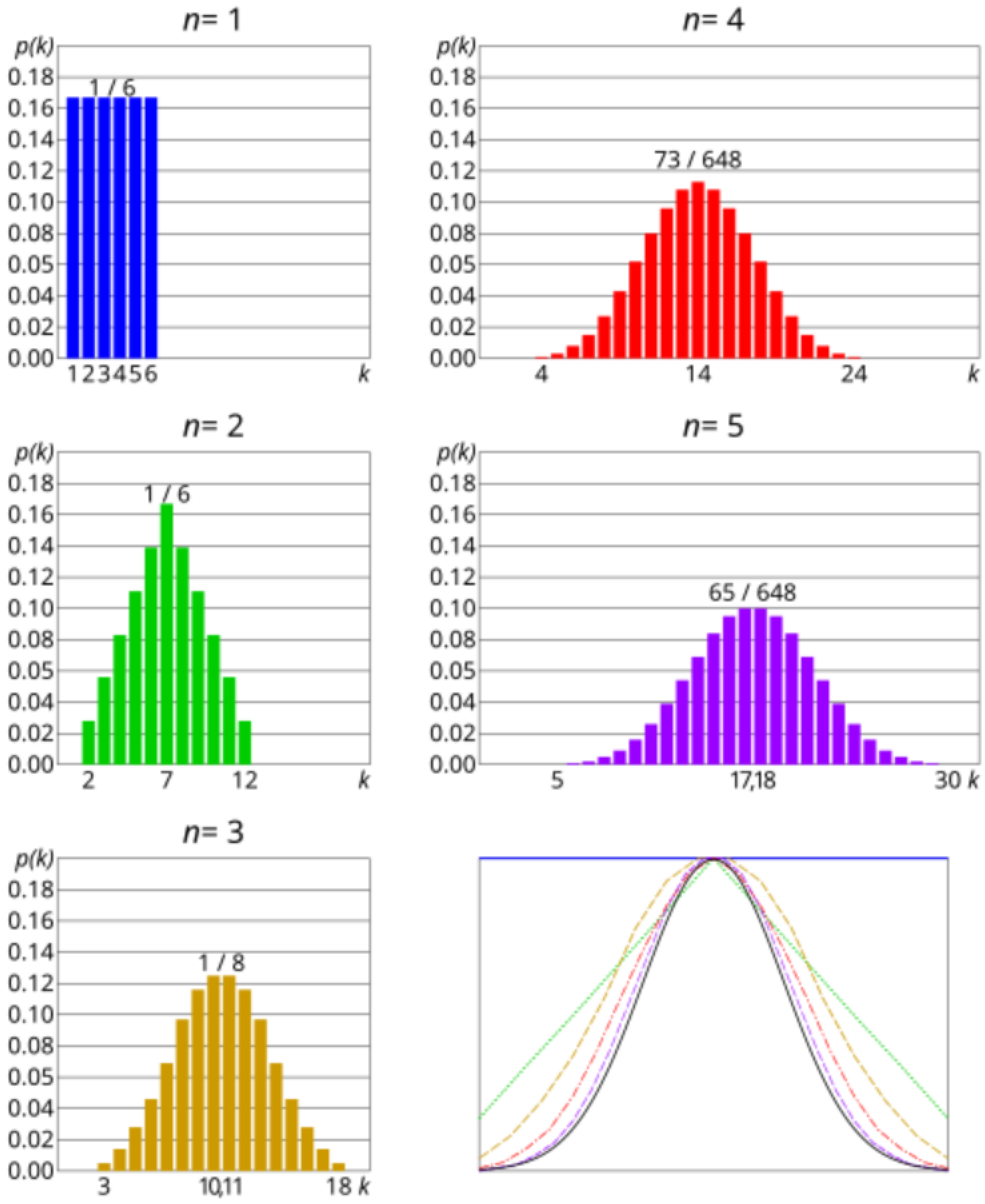
The Assumptions Underlying the Empirical Rule

Every "plus-or-minus such-and-such with 95% confidence" above rests on the **empirical rule**, and the empirical rule itself rests on two assumptions:

1. **The mean estimation error is zero** — the estimates are *unbiased*, not systematically too high or too low.
2. **The distribution of the errors in the estimates is normal.**

The second assumption invokes the familiar bell curve: for the standard version (mean 0, standard deviation 1) the curve peaks at zero, is symmetric, and falls off rapidly in both directions — its peak height is about 0.40, and it is essentially gone by $x = \pm 3$.

Why is normality a reasonable thing to assume for estimation errors? Because of the **central limit theorem**: the average of many statistically independent samples of a random variable with finite mean and variance is itself a random variable whose distribution converges to a normal distribution as the number of samples grows. Quantities built from the sum of many independent processes — measurement errors being the classic case — therefore tend to have nearly normal distributions. An estimated return averaged over thousands of independent spins is exactly such a quantity, which is what licenses treating its error as normal.



Source: Wikipedia, article "Normal distribution".

Defining Distributions: Discrete versus Continuous

A **probability distribution** captures the notion of the *relative frequency* with which a random variable takes on certain values. There are two flavors:

- **Discrete random variables** are drawn from a *finite* set of values — like the pockets of a roulette wheel. Life is easy here: you simply list the probability of each value, and those probabilities must add up to 1. For a roulette wheel, each pocket has probability $\frac{1}{37}$ or $\frac{1}{38}$ depending on the wheel, and they sum to one, as they must.
- **Continuous random variables** are drawn from the reals *between two numbers* — an infinite set of values. This case is trickier: you cannot enumerate a probability for each of infinitely many values. If you tried to assign a positive probability to each real number between 0 and 1, the sum would blow up. A different machinery is needed.

Probability Density Functions: Area, Not Height

That machinery is the **probability density function (PDF)**. Instead of asking for the probability of an exact value, we talk about the probability of the variable lying *between two values*. A PDF defines a curve over the range of the variable, and the punchline is:

> **The area under the curve between two points is the probability of the variable falling within that range** — not the height of the curve.

The PDF is nonnegative everywhere, and the total area under the whole curve equals 1, so the probability of landing somewhere in the possible values is 100%. Formally, for a random variable X with density f_X:

$$\Pr[a \le X \le b] = \int_a^b f_X(x)\,dx$$

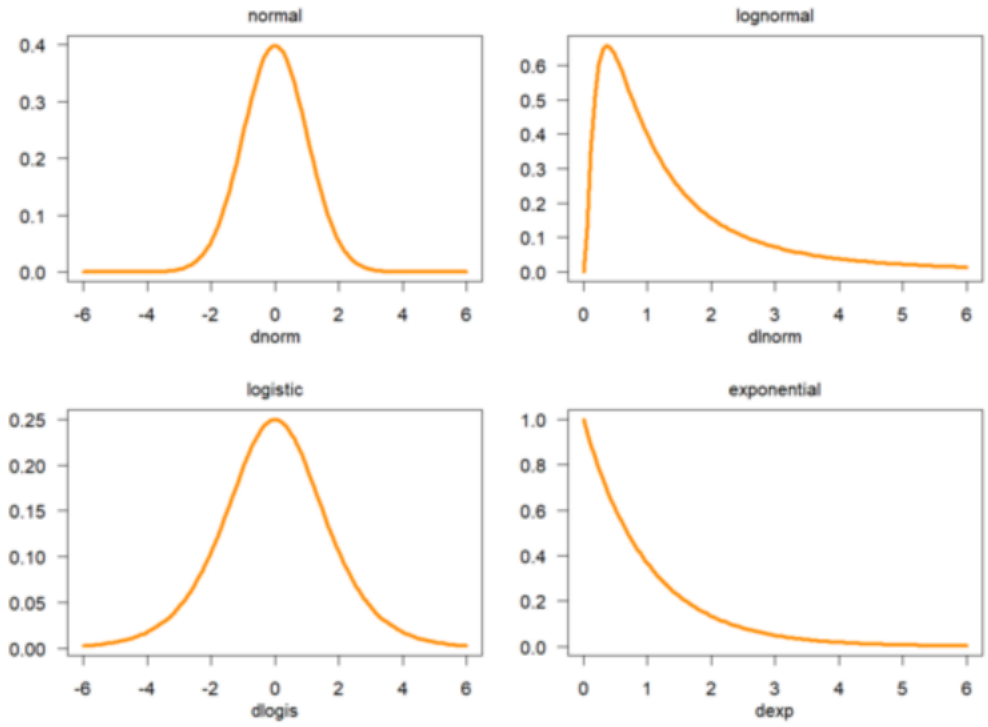
and the cumulative distribution function accumulates this from the left tail, $F_X(x) = \int_{-\infty}^{x} f_X(u)\,du$, with the density recovered as its derivative $f_X(x) = \frac{d}{dx}F_X(x)$ wherever that derivative exists. Intuitively, $f_X(x)\,dx$ is the probability of falling in the infinitesimal interval $[x, x + dx]$.

A key consequence: the absolute probability of a continuous variable taking on **any particular value is zero**. The classic illustration: suppose a bacterial species typically lives 20–30 hours. The probability that a bacterium dies at *exactly* 5.00… hours (measured with infinite precision) is zero — yet plenty of bacteria die near 5 hours. If the probability of dying between 5 and 5.01 hours is 0.02, then between 5 and 5.001 hours it is about 0.002, and between 5 and 5.0001 hours about 0.0002. The ratio

$$\frac{\text{probability of dying during an interval}}{\text{duration of the interval}} \approx 2\ \text{hour}^{-1}$$

is (approximately) constant — and that constant *is* the probability density at 5 hours, $f(5\,\text{hours}) = 2\ \text{hr}^{-1}$. Integrating f over any window of time gives the probability of dying in that window.

One terminological caution: the terms "probability distribution function" and "probability function" sometimes denote the PDF, but this usage is not standard among statisticians — those phrases may instead refer to the cumulative distribution function (CDF) or, for discrete variables, the probability mass function (PMF). The clean division: **PMF for discrete variables, PDF for continuous ones**; both are fundamental to statistical inference.



Source: Wikipedia, article "Probability density function".

The Normal Distribution and the Empirical Rule

The most famous PDF of all is the **normal (Gaussian) distribution**, a continuous distribution for a real-valued random variable with density

$$P(x) = \frac{1}{\sigma\sqrt{2\pi}}\, e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Don't panic about the formula — each symbol has a job:

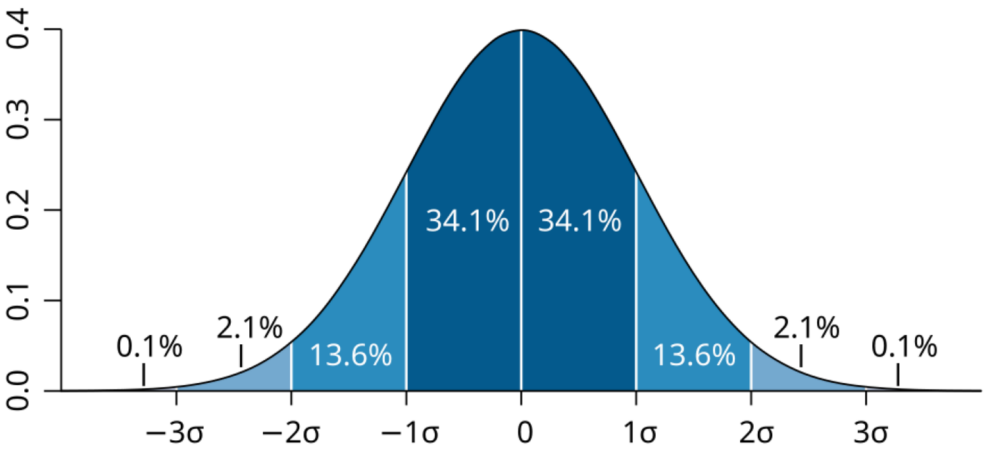
- μ is the **mean**: it tells you where the bell is centered (it is also the median and the mode);
- σ is the **standard deviation**: it tells you how wide the bell is;
- e is the constant defined as the infinite sum $\sum_{n=0}^{\infty} \frac{1}{n!}$.

The special case $\mu = 0$, $\sigma = 1$ is the **standard normal**, with density $\varphi(z) = \frac{e^{-z^2/2}}{\sqrt{2\pi}}$: peak value $\frac{1}{\sqrt{2\pi}} \approx 0.399$ at $z = 0$ (matching the "~0.40" seen in the plots), with inflection points at $z = \pm 1$. Every other normal distribution is just a stretched and shifted version of it: if Z is standard normal, then $X = \sigma Z + \mu$ is normal with mean μ and standard deviation σ; conversely $Z = (X - \mu)/\sigma$ standardizes any normal deviate, and the density picks up a factor $1/\sigma$ so the integral stays 1.

Here is why we care. For a normal distribution:

- roughly **68%** of the data falls within **one** standard deviation of the mean;
- roughly **95%** falls within **1.96** standard deviations of the mean — and that is *exactly* where the "plus-or-minus" number for 95% confidence comes from;
- roughly **99.7%** falls within **three** standard deviations of the mean.

(This is the **empirical rule**; using exact values, the fractions are 68.27%, 95.45%, and 99.73%.)



Source: Wikipedia, article "Normal distribution".

This closes the loop with the simulations: the empirical rule is what let us take the spread of the trial results and turn it into a confidence interval. When we reported the fair-roulette estimate as "+0.012% plus or minus 0.846% with 95% confidence," we meant precisely that — under the unbiasedness and normality assumptions, the true expected return lies within ±0.846% of the estimate with 95% probability, because 1.96 standard deviations of the error distribution captures 95% of its area.

A final caveat worth keeping in mind: despite its fame, the normal distribution is **frequently misused** in contexts where the data are not actually normally distributed, and it is a poor model there. Nor is "bell curve" a unique label — many other distributions (Cauchy, Student's t, logistic) share the bell shape.